NODEBASE - INTERVIEW PREPARATION

Implementation and Ownership

What I built, how the code is organised, and how I handled validation, errors, secrets, testing and deployment.

10 questions, answered in simple language

Each answer is written to be spoken in an interview. It is based on the actual codebase: Next.js 15, React 19, TypeScript, tRPC, Prisma, PostgreSQL, Better Auth, Polar, Inngest, BullMQ, Redis and React Flow.

Prepared in 2026

1. Which features did you implement yourself?

I implemented the full product end to end. On the backend, I designed the Prisma schema and built the tRPC routers for workflows, credentials and executions. I wrote the procedure chain for authentication and premium checks, the ownership guards, the workflow save transaction, the credential encryption helpers and the subscription helper. I also implemented the Inngest execution function, the topological sort, the executor registry and the node executors.

On the frontend, I built the dashboard layout, the workflow list, the credentials pages and the executions pages. I implemented the React Flow editor, including node selection, connection handling, the dirty-state tracking and the save button. I created the configuration dialogs for each node type, the node selector, the trigger panels and the live node status badges. I also built the authentication pages and forms.

I implemented the integration and operations work as well. That includes the Google Form, Stripe and Telegram webhook routes, the rate limiting and idempotency helpers, the BullMQ email and cron workers, the cron schedule picker and the queue dashboard. I configured Better Auth with email OTP, Google and GitHub login, and connected Polar for subscriptions and checkout.

Finally, I prepared the deployment setup. I wrote the Vercel build configuration and a multi-stage Dockerfile for the EKS path, added health and Sentry setup, and documented the environment variables. I also debugged the live email delivery and queue failures, which was a significant part of making the project actually work.

2. Explain one feature from frontend to database.

I will use the workflow save feature as the example. The user edits the canvas in the React Flow editor. React Flow keeps its own internal store, and the editor mirrors nodes and edges into React state. Whenever a non-selection change happens, the editor marks the workflow dirty with a Jotai atom. The Save button reads the current graph from the React Flow instance, not from the React state, and builds an object with the workflow id, nodes and edges.

The Save button calls the `useUpdateWorkflow` React Query hook, which calls the protected tRPC procedure `workflows.update`. On the server, Zod validates the input. The procedure then performs an ownership check with `findUniqueOrThrow`, scoped by the workflow id and the signed-in user id. That read also returns the existing node ids, which are needed to detect deletions.

The actual write happens inside a Prisma transaction. For each incoming node, the server upserts it by id, updates its position, type and name, and overwrites its JSON data only when the client sent non-empty data. Nodes that existed before but are missing from the payload are deleted, and their connections disappear through the database cascade. Connections are replaced as a set: the old rows are deleted and the new edges are created with the source and target node ids and handle names.

Finally, the transaction touches the workflow's `updatedAt` and returns. The client shows a success toast, invalidates the workflow queries, clears the dirty flag, and calls `syncCronSchedules` to reconcile Redis cron jobs with the saved graph. The database now contains the new graph, and the editor reflects the saved state.

3. How did you structure the project folders?

I used a feature-first structure under `src/features`. Each major product area, such as workflows, credentials, executions, editor, trigger, auth and subscriptions, owns its components, hooks and server code. This keeps related files together and makes it obvious where a change belongs. For example, everything about saving a workflow is under the workflows feature rather than spread across a global components folder and a global API folder.

Inside a feature, the server folder contains the tRPC router, the prefetch helpers and the URL parameter loaders. The hooks folder contains the React Query hooks. The components folder contains the UI for that feature. Some features also have a `params.ts` file for nuqs URL state. This pattern is consistent across workflows, credentials and executions, so a new developer can predict where things live.

Shared infrastructure lives outside the features. `src/app` contains the Next.js routes and API handlers. `src/trpc` contains the client, server helpers, context and router registry. `src/inngest` contains the client, the execution function, the utilities and the realtime channels. `src/lib` contains cross-cutting helpers such as database access, encryption, email, rate limiting, idempotency and subscriptions. `src/queue` contains the worker entry points.

The `src/components` folder contains shared UI components, including the shadcn primitives and the node selector. `src/config` contains constants and the node component registry. `prisma` contains the schema. I chose this structure because it scales better than grouping by file type. When the project grows, a feature can be understood, tested and even extracted without reading the whole repository.

4. How did you separate business logic from UI logic?

The main boundary is the tRPC router. The UI never talks to Prisma directly. It calls a procedure, and the procedure owns the authorization, validation, database access and error handling. This means a component can focus on rendering and user interaction, while the server code contains the rules. For example, the credential form does not decide whether the user has a subscription; the premium procedure does.

The executors are another clean separation. A node component is responsible for how the node looks and how its dialog collects configuration. The executor is responsible for what happens during a run. The executor receives data, context, step tools and a publish function, and it returns a new context. It does not know anything about React. That separation made it possible to add node types without turning the editor into a large collection of special cases.

Shared logic lives in `src/lib`, such as encryption, rate limiting, idempotency and subscription checks. These helpers can be used by routers, webhooks and workers without depending on the UI. The Inngest function is also independent of the frontend. It only knows the workflow id, the executor registry and the database.

There is still some UI logic in components, such as dialog state, form handling and dirty-state tracking. That is appropriate because it is presentation logic. The important rule is that business decisions, especially anything involving permissions, money, secrets or data integrity, never live only in a client component. The server always re-checks them.

5. How did you handle errors across the application?

At the tRPC layer, I use `TRPCError` with explicit codes. Unauthorized users get UNAUTHORIZED, users without a subscription get FORBIDDEN, and missing resources use the Prisma or tRPC not-found behaviour. On the client, mutations show a toast with a user-friendly message. Pages wrap suspense content in an error boundary, so a failed prefetch shows an error view instead of a blank screen.

In the workflow engine, errors are classified. Configuration mistakes such as a missing endpoint, variable name or credential throw `NonRetriableError`, because retrying will not fix them. Transient errors from external APIs are allowed to bubble up so Inngest can retry. Every executor follows the same pattern: publish loading, do the work, publish success, and publish error before rethrowing. The Inngest `onFailure` hook then marks the execution FAILED and stores the error message and stack trace.

Background workers have their own error handling. BullMQ retries failed email and cron jobs with exponential backoff and keeps a set of recent failures. The workers log failures to the console, and the queue dashboard makes them inspectable. The Google Form, Stripe and Telegram webhook routes use try/catch and return a JSON error with an appropriate status code, while still responding quickly to the provider.

I also use Sentry for unexpected errors and server-side logging for operational issues. The remaining improvement would be a central error mapper that converts known error types into consistent user messages, and alerts for failed executions and queue depth. Today the mechanisms exist, but they are not yet unified into one observability layer.

6. How did you validate user input?

Validation happens at several levels. On the client, forms use `react-hook-form` with a Zod resolver. For example, the credential form validates that the name and value are non-empty and that the type is one of the allowed credential types. The workflow dialogs validate the fields that each node type needs, such as the variable name, endpoint, method and prompt. This gives the user immediate feedback.

The server never trusts the client. Every tRPC procedure defines its own Zod input schema and re-validates the request. The credential router validates the name, the enum type and the value. The workflow router validates the node and edge shape, including the position object and the optional data record. The execution router validates pagination and search parameters. If the input is invalid, the request fails before any database write.

There is also validation at the database and type level. Prisma enforces the schema and the enums, and TypeScript ensures the code uses the generated types correctly. Ownership checks act as a form of authorization validation: a workflow or credential id alone is not enough, because the query also requires the current user id.

Some parts could be stricter. For example, `workflows.update` accepts `type` as a free string and only casts it to `NodeType` later, so an unknown type becomes a database error rather than a clean validation error. The empty-data rule is also a policy decision rather than pure validation. In a future version I would use the actual `NodeType` enum in the input schema and support an explicit way to clear configuration.

7. How did you test the project?

The project does not currently have an automated test runner, so I tested it through a combination of static checks and manual end-to-end verification. For every significant change I ran `npm run lint` to catch style and correctness issues, `npx tsc --noEmit` to check types, and `npm run build` to verify that the production build still worked. Those checks caught many problems before runtime.

For functional testing, I exercised the flows in the browser. I created accounts, verified email codes, signed in with Google and GitHub, created and saved workflows, configured credentials, ran workflows manually, triggered webhooks, and checked the execution history. I also tested the failure paths, such as a missing credential, a bad endpoint, a cyclic workflow and an expired verification code.

For infrastructure and data problems, I inspected the real state of the system. I queried PostgreSQL metadata to verify constraints and indexes, read live execution rows to compare success and failure behaviour, inspected BullMQ keys in Redis to understand waiting, active and failed jobs, and read the installed libraries when documentation was not enough. This was especially useful during the OTP email debugging.

If I had more time, I would add three layers of automated testing. Unit tests would cover executors, topological sort and validation schemas. Integration tests would run the tRPC routers and Prisma transactions against a temporary PostgreSQL database. End-to-end tests with Playwright would cover sign-up, workflow creation, save and execution. I would also add a test for duplicate webhook delivery and retry behaviour.

8. How did you manage environment variables and secrets?

Secrets are kept in environment variables and are never committed to the repository. The project includes an `.env.example` file to document the required variable names, while the real `.env` file is gitignored. The variables cover the database URL, Better Auth secret and URL, encryption key, SMTP credentials, OAuth client ids and secrets, Polar token, Inngest keys, Upstash URLs and the public app URL.

Only variables that are safe for the browser use the `NEXT_PUBLIC_` prefix. In practice that is the application URL, which is needed to build webhook and OAuth links. Everything else stays on the server. The tRPC client never receives database credentials, the encryption key or provider API keys. That separation is important because any `NEXT_PUBLIC_` value ends up in the JavaScript bundle.

User-supplied third-party credentials are not stored in environment variables. They are stored in the Credential table and encrypted with Cryptr using the `ENCRYPTION_KEY`. Plaintext is decrypted only inside an executor, immediately before it is passed to the provider SDK, and it is never written to an execution row or sent to the client. The executors also scope credential lookups by user id, so a user cannot use another user's credential by guessing its id.

There are still improvements to make. The encryption key is a single secret with no rotation path, so losing it would make every credential unreadable. In production I would move secrets into a managed secret store such as AWS Secrets Manager, add key rotation, restrict access by role, scan for accidentally committed secrets, and separate development, staging and production credentials. The current setup is appropriate for development but would need hardening for a larger deployment.

9. How did you deploy the application?

The project is prepared for two deployment paths. The Vercel path uses the `vercel-build` script, which generates the Prisma client and then runs the Next.js production build. The Inngest function is served from `/api/inngest`, the auth routes from `/api/auth`, and the tRPC routes from `/api/trpc`. PostgreSQL is hosted on Neon, Redis on Upstash, email through SMTP, billing through Polar, and execution through Inngest Cloud.

The second path is a container deployment for AWS EKS. I wrote a multi-stage Dockerfile that installs dependencies, generates Prisma, builds Next.js, and produces a small runtime image. It runs as a non-root user, copies the standalone Next output, and includes the Prisma CLI and dotenv tools needed at runtime. The deployment also needs the email worker and cron worker running as separate processes, because they must not share the web server lifecycle.

Locally, `mprocs` starts the Next server, the Inngest dev server, the email worker, the cron worker and the ngrok tunnel. The ngrok tunnel is needed because webhooks and OAuth callbacks require a public URL. The production deployment would use a stable domain instead, which also avoids breaking OAuth callbacks when the tunnel URL changes.

The EKS migration is still in progress. The remaining work includes setting up Kubernetes manifests or Helm charts, managing secrets, configuring health checks and resource limits, adding CI/CD for migrations and images, and verifying the dotenv behaviour in the runtime container. So the application is deployable, but the production infrastructure is not yet fully finished.

10. How would you demonstrate that you understand the code you wrote?

I would start by tracing one request from the browser to the database and back. For example, I would show how the Execute button calls the tRPC procedure, how the event reaches Inngest, how the execution row is created, how topological sort orders the nodes, how an executor reads the context and decrypts a credential, and how the final output is stored and displayed. Being able to follow that path proves that I understand the system rather than just individual files.

Next, I would explain the design decisions and their trade-offs. I would say why PostgreSQL instead of MongoDB, why Inngest instead of a plain queue for execution, why nodes are merged instead of deleted and recreated, and why connections are replaced as a set. I would also be honest about the weaknesses, such as the missing automated tests, the `INITIAL` node bug, the OpenAI channel mix-up and the lack of cycle validation at save time.

I would then debug something live. I could show how to inspect a failed execution, read the error from the database, check the Inngest run, inspect a BullMQ failed job, or verify a database constraint. I would also show how I read the installed library source when the behaviour was not clear, which is how I confirmed details about Better Auth, BullMQ and the React Flow diff mechanism.

Finally, I would demonstrate a change. For example, I could add a small node type or fix one of the known bugs, run lint, typecheck and build, and explain how the change moves through the five touchpoints: schema, channel, executor, node component and dialog. That shows both understanding and the ability to work safely in the codebase.